

perusal, and is backed by a set of tools that allow the reporting and tracking of requests and bugs.

Since the first principles of software engineering are somewhat generic, an intense level of collaboration is reflected in the ability to reuse innovations at an intra-project and inter-project level.

If you collaborate to innovate results in short development sprints, it ensures that a prototype can be released early. And, since development happens within the boundaries of predefined milestones, releases happen often. This ensures that the end users have access to gradually improving versions of the application, with direct inputs into the development lifecycle and in feature enhancements.

So far so good! But what makes all this happen? Infrastructure and communication.

The primary infrastructure requirement that needs to be planned, prepared, configured and deployed, is a 'forge' or a collaboration platform. By providing the features of version control, forums, group mailing (or, mailing lists), issue tracker, etc., a collaboration platform provides an ideal way to initiate constant communication. Having a central repository of codebases (i.e., the source code along with means to tag and search) makes it easy for application of design patterns on code. It also facilitates peer review and thus leads to improved code strength.

A forge provides the ideal set of data points that make it possible for teams working on modules and projects to meet, brainstorm and discuss—both in person, as well as virtually. The world of open source makes it very normal to hold code review and stand up meetings virtually, using virtual whiteboard technologies like XMPP, Obby, etc. Coupling such communication infrastructure units with an established policy for software development makes for a win-win scenario while adopting the open source model.

The government of India has initiated a large number of citizen-centric IT initiatives with the objective of making available services to the common man. These initiatives are based on IT design patterns that share commonalities. Identification of such 'common' aspects and adopting an open source software development model using the appropriate infrastructure would lead to a set of innovation patterns.

The open source model can also be extended to projects beyond the citizen-centric initiatives and at the level of the electronic 'mart' projects being undertaken for agriculture. Since the model allows collaboration, it brings in an additional number of software developers towards reviewing code and fixing issues. Additionally, it allows for rapid prototyping, thus providing benefits of early release and testing of software. The model also lends itself well to issues of standard definition, font creation, auditing of Web services for security and standards compliance, among other things. The underlying theme is to not limit the open source model towards software development, but to extend, adapt and adopt it to as many aspects of the work

flow as is permitted, with the end objective of producing high-quality software and content.

The open source model for software development is not limited to putting in place technology infrastructure—hardware and software. It also requires that an adequate definition of the software development policy is in place. Such a policy would include methods to check and control, that provide the project administrators with a granular view of the project's progress. Additionally, this policy would encourage code review, code reuse and enhancements to existing codebases.

The reason for such a model to attain a measure of success would be because it does not mandate a complete overhaul of the existing process. The open source model for software development puts in place a system of project management that allows innovation to be transparent, prototyping to be rapid and collaboration to be constant. A resultant effect of approaching software development using aspects of open source is a higher degree of collaboration by leveraging the effect of 'crowd-sourcing'. In effect, doing code development in the 'incrementally improving' method allows for a greater degree of rigour in using test cases. Such a course of action would create improved software. The ability to prototype rapidly also provides the scope of customisation.

The constant availability of a 'forge' or repository of projects and associated codebases provides a unique opportunity to estimate the maturity of a software development unit. The added advantage of reuse allows a metadata component to be added to such estimations. It is somewhat logical that robust codebases and strong design patterns would be adopted across the organisation. However, such adoptions do not lead to a monoculture of coding patterns. Instead, it allows creativity to flourish within the scope of coding standards and project guidelines as defined by the policy formulating entities. The repository provides a replacement to unorganised institutional memory and, in a way, provides a historical timeline of projects as they evolved through their project management charter.

To sum up, adopting and adapting the open source model of software development for government-funded projects provides a win-win scenario for all stakeholders. The developers are attuned to the idea of rapid prototyping and release, which makes for an increased involvement from the consumers/customers of the software. And the resulting feedback loop increases the robustness, efficiency and innovation factor of the software projects. Collaborating to innovate provides the bedrock for a much shorter time-to-release in the software development lifecycle of a software project. **END** 

By: Sankarshan Mukhopadhyay

The author works at Red Hat and has been using Fedora since the days of Fedora Core 1. He can be reached at sankarshan@redhat.com or at sankarshan@jabber.org (instant messaging).